

CIMA

Control Interno y Monitoreo de Adultos Mayores

Especificación técnica maestra

25 de septiembre de 2026

Índice

Especificación técnica maestra	1
Control de cambios	2
1. Propósito y alcance	2
2. Estado actual comprobado	2
3. Stack tecnológico	2
4. Principios de desarrollo	3
5. Fase 0 — Fundación	3
6. Fase 1 — Residentes	3
7. Fase 2 — Ficha del residente	3
7.1 Primer corte implementado	3
7.2 Evolución funcional	4
8. Fase 3 — Habitaciones y estadías	4
9. Fase 4 — Alimentación	4
10. Información médica	4
11. Fase 5 — Medicamentos	4
12. Fase 6 — Estadías, cargos y pagos	4
13. Fase 7 — Documentos	5
14. Fase 8 — Usuarios y autenticación	5
15. Fase 9 — RBAC y permisos	5
16. Fase 10 — Auditoría	5
17. Fase 11 — Dashboard	5
18. Modelo de datos objetivo	5
19. API objetivo	6
20. Seguridad	6
21. Accesibilidad	7
22. Testing	7
23. Definición de terminado	7
24. Privacidad	7
25. Roadmap de versiones	7
26. Secuencia de implementación	8
Próximo incremento recomendado	8

Especificación técnica maestra

Versión del documento: 0.4

Fecha: 25 de septiembre de 2026

Estado: Fundación operativa; CRUD de residentes y primer corte de ficha personal implementados.

Versión de software de referencia: v0.3 (base publicada); esta revisión técnica describe cambios posteriores todavía sin release/tag.

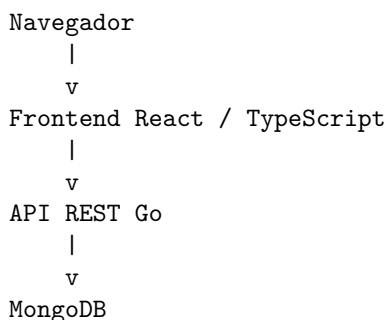
Control de cambios

- **0.2, 24-09-2026:** especificación base y roadmap inicial.
- **0.3, 25-09-2026:** estado contrastado con el código y Docker; se registra la ficha personal/contactos, sus endpoints, validaciones y pruebas ejecutadas. Se aclaran las funciones todavía pendientes.
- **0.4, 25-09-2026:** CRUD de residentes completado; incluye consulta individual, edición parcial, archivado lógico, timestamps, acciones de interfaz y pruebas unitarias/de integración.

1. Propósito y alcance

CIMA es una aplicación web interna e independiente para apoyar la gestión de una residencia de adultos mayores. Su objetivo es centralizar, de forma progresiva, la operación de residentes, habitaciones, estadías, alimentación, información médica, medicamentos, documentos, cargos, pagos, usuarios, permisos, auditoría y dashboards operacionales.

La arquitectura mantiene una separación estricta entre navegador, API y base de datos. El navegador nunca se conecta directamente a MongoDB.



2. Estado actual comprobado

El entorno Docker Compose contiene MongoDB, backend Go y frontend React. Go, Node.js y MongoDB se ejecutan en contenedores; no se requieren instalados en el computador anfitrión.

Área	Estado actual
Infraestructura	<code>docker compose up --build</code> construye y levanta MongoDB, backend y frontend. MongoDB conserva
Healthcheck	<code>GET /health</code> responde <code>{"status":"ok"}</code> .
Residentes	<code>GET/POST /api/v1/residents</code> , <code>GET/PATCH/DELETE /api/v1/residents/{id}</code> . <code>DELETE</code> archiva lógic
Ficha personal	<code>GET /api/v1/residents/{id}/profile</code> y <code>PATCH /api/v1/residents/{id}/profile</code> están impleme
Interfaz	Crear/listar residentes, seleccionar uno, cargar su ficha y editar datos personales y contactos. Hay est
Validaciones API	IDs MongoDB válidos; nombres no vacíos; estado limitado a <code>active/inactive</code> para creación/edición;
Timestamps	Creación y actualización registran <code>createdAt</code> y <code>updatedAt</code> ; archivado registra <code>archivedAt</code> . Los times
Interfaz	Crear/listar residentes, consultar ficha, editar nombre/apellido y archivar con confirmación. El registr
Verificación ejecutada	Build de ambas imágenes Docker; pruebas unitarias Go; ciclo real <code>POST/GET/PATCH/DELETE</code> co

Todavía no implementado: filtros, búsqueda, paginación, autenticación, autorización, habitaciones, alimentación, datos médicos, medicamentos, auditoría, pagos, documentos y dashboards. Hay pruebas unitarias de validación/JSON y verificación manual de integración con MongoDB; falta una suite automatizada de integración API/MongoDB y pruebas E2E.

La información médica no forma parte de esta primera ficha. La API actual aún no tiene autenticación ni RBAC y debe tratarse como entorno de desarrollo, no como sistema apto para datos reales o producción.

3. Stack tecnológico

- **Frontend:** React, TypeScript y Vite. Se prevén TanStack Query, React Hook Form y Zod a medida que crezca el módulo.

- **Backend:** Go, Chi, driver oficial de MongoDB, Argon2id, JWT y Zap conforme se implementen autenticación, seguridad y logging.
- **Base de datos:** MongoDB.
- **Infraestructura:** Docker, Docker Compose y, para despliegue, Nginx o Caddy.
- **Calidad:** Vitest, Playwright, `go test`, Testify y GitHub Actions.
- **Documentación de API:** OpenAPI / Swagger.

Las tecnologías listadas como previstas no deben interpretarse como dependencias ya implementadas.

4. Principios de desarrollo

1. Construir en incrementos verticales pequeños y verificables.
2. Mantener Docker Compose como entorno principal; no requerir Go, Node ni MongoDB locales.
3. Mantener `.env.example`; no guardar secretos en Git.
4. Validar en frontend para una buena experiencia y volver a validar siempre en backend.
5. Separar datos personales, operacionales, médicos y económicos según necesidad de conocer.
6. No crear colecciones anticipadamente: incorporar cada módulo cuando exista una necesidad funcional comprobada.
7. Preservar `docker compose up --build` como criterio transversal.

5. Fase 0 — Fundación

Objetivos: Docker Compose, `.env.example`, secretos fuera del repositorio, servicios Go/Node/MongoDB en contenedores, convenciones de código, lint, format, pruebas automatizadas y `/health`.

Criterio de término: `docker compose up --build` levanta MongoDB, backend y frontend. La compilación actual de frontend y backend ocurre dentro de sus imágenes. Los criterios de lint y pruebas automatizadas todavía requieren completarse.

6. Fase 1 — Residentes

Colección: `residents`.

Implementado: crear, listar, obtener por ID, actualizar parcialmente y archivar residentes. Campos iniciales: `id`, `firstName`, `lastName`, `status`, `createdAt`, `updatedAt`; al archivar se registran `archivedAt` y `status`: "archived". MongoDB genera el ID. El backend recorta espacios, rechaza nombres vacíos, limita el estado de creación/edición a `active/inactive` y rechaza campos JSON desconocidos.

Pendiente:

- Búsqueda, filtros, paginación, ordenamiento y estados definidos.
- Pruebas automatizadas de integración API/MongoDB y E2E. Las pruebas de ciclo CRUD realizadas hasta ahora son manuales.

La API debe validar siempre, aunque el frontend ya valide los datos.

7. Fase 2 — Ficha del residente

La ficha es el punto de acceso operativo al residente, pero no una autorización para reunir toda la información en una sola respuesta. Las áreas sensibles deben tener permisos y endpoints separados.

7.1 Primer corte implementado

La ficha se almacena como subdocumento `profile` en el documento MongoDB del residente existente. No se crea una colección independiente para este primer corte.

Campos personales y de contacto implementados:

- Fecha de nacimiento (`dateOfBirth`), opcional, formato YYYY-MM-DD.
- Teléfono y correo del residente, opcionales.

- Contacto principal: nombre, relación, teléfono y correo.
- Contacto de emergencia: nombre, relación, teléfono y correo.

Endpoints:

GET /api/v1/residents/{id}/profile

PATCH /api/v1/residents/{id}/profile

PATCH actualiza solo los campos presentes, actualiza `updatedAt`, rechaza un ID inválido o residente inexistente y no acepta propiedades desconocidas. Los campos de fecha y correo se validan en backend. La interfaz ofrece carga, edición, guardado, errores y confirmación; no habilita el formulario antes de cargar correctamente los datos.

7.2 Evolución funcional

Los datos adicionales se incorporarán cuando estén definidos por necesidad operacional y minimización de datos. El modelo completo contempla secciones conceptuales de identificación, datos personales, contactos, alojamiento, alimentación, información médica, medicamentos, estadía, economía, documentos e historial/auditoría.

La habitación, estadía, alimentación, información médica, medicamentos, economía y documentos se implementarán como módulos con contratos y controles propios. La información médica debe tener permisos específicos y no debe mezclarse con la respuesta general de perfil.

8. Fase 3 — Habitaciones y estadías

Colecciones previstas: `rooms`, `stays`.

Habitaciones: número/código, capacidad, estado, ocupación y observaciones. Estadías: ingreso, permanencia, salida, estado y motivo de salida. Debe existir historial de cambios de habitación y el sistema debe impedir asignaciones incompatibles con la capacidad y las reglas operativas que se definan.

9. Fase 4 — Alimentación

Módulo propio para preferencias, alergias, restricciones, indicaciones y observaciones. La vista de cocina expondrá solo lo necesario para preparar y entregar alimentos; no concederá acceso automático a información médica completa.

10. Información médica

Se prevé un módulo independiente para antecedentes relevantes, indicaciones y observaciones operacionales. Su acceso tendrá controles específicos por permiso y necesidad de conocer. No incluir secretos ni datos médicos innecesarios en logs.

11. Fase 5 — Medicamentos

Colecciones previstas: `medications`, `medication_events`.

Una prescripción podrá incluir medicamento, dosis, unidad, vía, frecuencia, horario, inicio, término e indicaciones. Los eventos registrarán administración, omisión u otro resultado, motivo, fecha/hora y usuario responsable. Los eventos relevantes deben ser trazables y no editables libremente como datos de estado ordinarios.

12. Fase 6 — Estadías, cargos y pagos

Colecciones previstas: `stays`, `charges`, `payments`.

Los cargos contemplarán concepto, monto, fecha, estado, residente y referencia. Los pagos contemplarán monto, fecha, medio, referencia, usuario y estado. Los datos económicos tendrán permisos distintos de los datos médicos.

13. Fase 7 — Documentos

Colección prevista: `documents` para metadatos. Los archivos se alojarán en almacenamiento privado, nunca en una ruta pública de documentos médicos. La descarga será autorizada por API y podrá usar URLs temporales. Se auditarán carga, consulta y eliminación.

14. Fase 8 — Usuarios y autenticación

Colección prevista: `users`.

Roles iniciales propuestos: `admin`, `manager`, `nurse`, `kitchen`, `reception` y `accounting`. La autenticación incluirá login, access token de vida corta, refresh token, expiración, revocación y logout. Las contraseñas se protegerán con Argon2id; nunca se guardarán contraseñas en texto plano ni hashes débiles.

15. Fase 9 — RBAC y permisos

El rol no será suficiente por sí solo. Se usarán permisos granulares, por ejemplo:

<code>resident.read</code>	<code>resident.create</code>	<code>resident.update</code>
<code>medical.read</code>	<code>medical.create</code>	<code>medical.update</code>
<code>medication.read</code>	<code>medication.administer</code>	
<code>food.read</code>	<code>food.update</code>	
<code>billing.read</code>	<code>billing.create</code>	<code>billing.update</code>
<code>audit.read</code>		

El frontend puede ocultar opciones, pero la autorización real siempre ocurre en el backend. Una llamada directa no autorizada debe ser rechazada por la API.

16. Fase 10 — Auditoría

Colección prevista: `audit_logs`.

Según la sensibilidad de cada operación, registrar actor, acción, recurso, identificador del recurso, timestamp y contexto mínimo. Ejemplos: creación/actualización/archivo de residente, cambios médicos, prescripciones, administración de medicamentos y pagos. Nunca registrar contraseñas, tokens, secretos ni información médica innecesaria.

17. Fase 11 — Dashboard

Las vistas dependerán del rol y mostrarán solo la información necesaria: administración (residentes, habitaciones, usuarios y alertas), enfermería (medicación y alertas autorizadas), cocina (restricciones, alergias y preferencias), recepción (ingresos, salidas y habitaciones) y contabilidad (cargos, pagos y saldos).

18. Modelo de datos objetivo

Colecciones previstas, a introducir progresivamente:

```
users
residents
rooms
medications
medication_events
medical_records
documents
stays
charges
payments
audit_logs
```

En el estado actual, **residents** contiene identidad básica y el subdocumento **profile** para los campos personales/contactos descritos en la sección 7.1.

19. API objetivo

Base: /api/v1.

```
GET    /health
GET    /residents
POST   /residents
GET    /residents/{id}
PATCH /residents/{id}
DELETE /residents/{id}          # baja lógica
GET    /residents/{id}/profile
PATCH /residents/{id}/profile
GET    /residents/{id}/food
PATCH /residents/{id}/food
GET    /residents/{id}/medical-records
POST   /residents/{id}/medical-records
PATCH /residents/{id}/medical-records/{recordId}
GET    /residents/{id}/medications
POST   /residents/{id}/medications
PATCH /residents/{id}/medications/{medicationId}
GET    /medication-events
POST   /medication-events
GET    /rooms
POST   /rooms
PATCH /rooms/{id}
GET    /stays
POST   /stays
PATCH /stays/{id}
GET    /residents/{id}/documents
POST   /residents/{id}/documents
DELETE /residents/{id}/documents/{documentId}
GET    /charges
POST   /charges
PATCH /charges/{id}
GET    /payments
POST   /payments
GET    /audit-logs
GET    /dashboard
POST   /auth/login
POST   /auth/refresh
POST   /auth/logout
```

Están implementados GET /health, GET/POST /residents, GET/PATCH/DELETE /residents/{id} y GET/PATCH /residents/{id}/profile. Las demás rutas son objetivo, no funcionalidad disponible.

20. Seguridad

Requisitos transversales para antes de producción:

- Argon2id, JWT de vida corta y refresh tokens revocables.
- HTTPS, CORS restrictivo, rate limiting y cabeceras de seguridad.
- Validación backend y DTOs estrictos; límites de tamaño de request.
- Consultas MongoDB construidas explícitamente; protección contra NoSQL injection.
- Protección XSS y CSRF cuando corresponda.

- Secretos fuera del repositorio; configuración de producción segura.
- Autorización por permiso y recurso.
- No exponer información médica en logs.
- Documentos privados, backups protegidos y restauración probada.

El código JavaScript inspeccionable no es una barrera de seguridad. La seguridad depende de backend, autenticación, autorización, validación y protección de datos.

21. Accesibilidad

Objetivo **WCAG 2.2 AA**: teclado, foco visible, orden lógico, labels, errores accesibles, contraste, no depender solo del color, lectores de pantalla, diseño responsive/mobile-first, targets táctiles adecuados, `prefers-reduced-motion`, modos claro/oscuro y zoom nativo al 200 % sin perder funcionalidad.

22. Testing

- **Unitario**: `go test` para backend y Vitest para frontend; validadores, reglas, servicios y componentes críticos. Hay cobertura Go para estados y decodificación JSON estricta.
- **Integración**: API con MongoDB: creación, lectura individual, actualización, archivado, validaciones y errores. El CRUD se verificó manualmente; automatizar esta suite sigue pendiente.
- **E2E**: Playwright para crear, consultar y editar residente; validar formularios; login; permisos; ficha; medicamentos y pagos.
- **CI**: builds y pruebas reproducibles mediante Docker/GitHub Actions.

Casos adicionales para la ficha: ID inválido, residente inexistente, fecha/correo inválidos, campo desconocido, actualización parcial, lectura tras escritura y error de carga sin habilitar guardado vacío.

23. Definición de terminado

Un módulo no está terminado solo porque funciona manualmente. Debe contemplar modelo, endpoint, validación, autorización, interfaz responsive y accesible, estados de carga/vacío/éxito/error, auditoría cuando corresponda, pruebas y documentación. `docker compose up --build` debe continuar funcionando.

24. Privacidad

La información personal y médica es sensible. Aplicar mínimo privilegio, necesidad de conocer, recopilación mínima, trazabilidad, retención y eliminación definidas, y backups protegidos. No usar datos reales en GitHub, fixtures públicos, capturas ni entornos de desarrollo sin controles adecuados.

Antes de producción se debe revisar la normativa legal y regulatoria aplicable en Chile. La implementación actual no incorpora todavía autenticación ni permisos y no debe usarse con información real de residentes.

25. Roadmap de versiones

- 0.1.x Fundación + API + frontend + residentes
- 0.2.x Ficha completa del residente
- 0.3.x Medicamentos + eventos
- 0.4.x Habitaciones + estadías + alimentación
- 0.5.x Usuarios + RBAC + auditoría + seguridad
- 0.6.x Cargos + pagos + documentos
- 0.7.x Dashboard por rol
- 0.8.x Testing integral + accesibilidad + hardening
- 1.0.0 MVP de producción

La numeración es objetivo de roadmap; la versión desplegada debe actualizarse explícitamente al preparar cada release.

26. Secuencia de implementación

Paso	Trabajo	Estado al 25-09-2026
1	Terminar CRUD de residentes	Implementado: creación, listado, lectura individual, edición y baja
2	Modelo de ficha completa	Parcial: subdocumento personal/contactos; los módulos sensibles
3	<code>GET/PATCH /residents/{id}/profile</code>	Implementado y probado contra MongoDB.
4	Pantalla de ficha seleccionando residente	Primer corte implementado desde la lista; ruta dedicada <code>/residents/{id}</code>
5	Datos personales y contactos	Primer conjunto implementado: nacimiento, teléfono, correo y contraseña
6	Habitación	Pendiente.
7	Alimentación	Pendiente y separado de información médica.
8	Antecedentes e indicaciones médicas	Pendiente, con acceso sensible específico.
9	Separar visual y técnicamente información sensible	Principio definido; implementación de permisos pendiente.
10	Medicamentos	Pendiente.
11	Usuarios y permisos	Pendiente.
12	Auditoría	Pendiente.
13	Estadías	Pendiente.
14	Cargos y pagos	Pendiente.
15	Documentos privados	Pendiente.
16	Dashboards por rol	Pendiente.
17	Testing integral	Parcial: pruebas unitarias Go y verificaciones manuales CRUD/MongoDB
18	Hardening de seguridad	Pendiente.
19	Backups y restauración	Pendiente.
20	Preparación para producción	Pendiente.

Próximo incremento recomendado

Automatizar las pruebas de integración API/MongoDB para residentes y ficha, y después avanzar con alojamiento (habitaciones/estadías) según las reglas operativas acordadas. Mantener Docker como vía de ejecución y no cargar datos reales hasta disponer de autenticación, autorización y medidas de privacidad adecuadas.